The Warm Icing Model

Data provenance and fact-check guide

This document lists every outside source the project uses, what was taken from each, how the synthetic benchmark data was generated from those inputs, the code that does it, and the checks added against fitting our own model. Every number is traceable to a file in the project package (`warm-icing-model`); file paths are given relative to that folder. Items still worth checking by hand are collected in Section 7.

1. References used

Only one source supplies measured warpage data (R1). The others supply material properties, geometry, measurement-noise levels, method or motivation. No public dataset pairing measured warpage maps with known copper layouts was found; substrate and package makers treat this data as confidential.

Ref	Source	What we took from it	Used in	Check status
R1	Tsai, Wang & Liu, "Thermally-Induced Deformations and Warpages of Flip-Chip and 2.5D IC Packages Measured by Strain Gauges", Materials 14:3723 (2021), doi:10.3390/ma14133723	Measured shadow-moiré corner warpage of a flip-chip package, 25–260 °C (Fig. 13, read by eye). Table 1 materials: Si 131 GPa / 0.28 / 2.8 ppm; bump/underfill 7,600 → 100 MPa and 29 → 98 ppm/K across Tg 120 °C; substrate 26 GPa / 0.39 / 14.4 ppm. Zero warpage near 120 °C. Moiré sensitivity 12.7 µm/fringe.	validate_flipchip.py; tsai2021_fig13_flipchip_digitized.csv; benchmark core material and 120 °C stress-free temperature (benchmark_config.json)	Table 1 values checked against the PDF text. Fig. 13 values read off the figure (approximate).
R2	Cadence white paper, "Thermal Warpage Simulation of a Temperature-Dependent Linear Elastic Material Package"	Model geometry of the R1 flip-chip package (Fig. 3): substrate 31 × 31 × 1.08 mm, bump/underfill layer 0.07 mm, die 10 × 10 × 0.785 mm; materials (Table 3) matching R1 Table 1.	validate_flipchip.py	The PDF is image-only, so these values could not be text-searched here: check Fig. 3 and Table 3 by eye. Square footprints were assumed.
R3	Kulterman, Loh, Fu & Tsuriya, "Comparison of Advanced Package Warpage Measurement Metrologies", iNEMI, IEEE IEMT Conference, 2016	Shadow-moiré theoretical Z resolution 0.85–1.27 µm; flatness standards measured within ~1–3 µm of theory (Table IV: 670.3 vs 668, 1,024.6 vs 1,025.6, 661 vs 662 µm); shadow moiré read ~20 µm (room temperature) and ~40 µm (peak) lower than other tools on the same package.	Benchmark noise levels (benchmark_config.json, measurement_noise_reference); stress tests S3 (tool bias) and S4 (3 µm noise)	Resolution, flatness and the 20/40 µm offsets checked in the text. The package's total warpage swing (used to turn 20–40 µm into a ~10% scale bias) still needs reading off its Fig. 11.
R4	Panigrahy, Che, Ong, Ng & Kumar (Micron), "Deep Learning Study on Memory IC Package Warpage Using Deep Neural Network and Finite Element Simulation", Chips 4:35 (2025), doi:10.3390/chips4030035	Precedent: industry builds warpage datasets from finite-element simulation (a design-of-experiments database) when measured data is scarce.	Justification for the synthetic benchmark (Section 3)	Context only; no numbers taken.
R5	Silk, Wenger, Coyle, Goodbread & Giamis, "Double Reflow-Induced Brittle Interfacial Failures in Pb-free Ball Grid Array (BGA) Solder Joints" (conference paper; year not printed, references run to 2009)	Motivation: warpage-induced stress opened solder joints of a 31 mm BGA; warpage measured by thermal shadow moiré.	Problem statement only	Context only; no numbers taken.
R6	Layden et al., "Quantum-enhanced Markov chain Monte Carlo", Nature 619, 282 (2023)	The quantum proposal: time evolution under (1−γ)·α·H_Ising + γ·ΣX from the current state, then a classical Metropolis accept step; random γ and t ensemble.	run_benchmark.py (quantum_Q), main.py (proposal_circuit), hardware circuits	Method; settings listed in Section 5.
R7	ABF GX-T31 build-up film (Ajinomoto) as reported in secondary summaries	Nominal build-up layer: E 7.5 GPa, CTE 23 ppm/K (25–150 °C). Poisson ratio 0.30 assumed.	benchmark_config.json (buildup_nominal)	Not yet confirmed against the Ajinomoto datasheet (Section 7).
R8	Handbook values for copper	E 117 GPa, ν 0.34, CTE 17 ppm/K	benchmark_config.json (copper)	Standard values.

2. Real measured data used

The only measured warpage data is R1's Fig. 13: corner warpage (point c relative to the centre) of the flip-chip package at 13 temperatures. It was read off the published figure by eye and is stored in `benchmark/tsai2021_fig13_flipchip_digitized.csv`. It is used for one purpose: to check that the forward physics (our 3D finite-element model) reproduces a real package. It is not used to build or tune the benchmark.

T (°C)	Moiré, heating (µm)	Moiré, cooling (µm)	Paper 3D FEM (µm)	Our 3D FEA (µm)
25	-58	-58	-58	-59.2
40	-50	-49	-49	-49.8
60	-38	-45	-37	-37.4
80	-28	-34	-24	-24.9
100	-18	-28	-12	-12.5
120	0	0	0	0.0
140	12	21	4	4.1
160	12	23	8	8.1
180	14	20	12	12.2
200	17	27	16	16.2
220	19	22	20	20.3
240	22	26	24	24.4
260	27	–	28	28.4

Result (benchmark/validation_tsai2021_flipchip.txt): −59.2 µm vs −58 µm measured at 25 °C; 3.2 µm RMS against the heating moiré curve over an 85 µm range; about 0.5 µm RMS against the paper's own FEM. All inputs come from R1 Table 1 and R2 geometry; nothing was fitted to the curve. Below 120 °C the underfill uses its below-Tg values (7,600 MPa, 29 ppm/K) and above 120 °C its above-Tg values (100 MPa, 98 ppm/K); warpage is scaled linearly with (T − 120 °C) within each regime.

3. Synthetic data we generated

3.1 Why synthetic data

Scoring an inverse method needs the true answer. Real boards never come with their copper layout labelled, and measured warpage maps with known layouts are not public. The standard approach, also used in industry (R4), is to generate measurements from a trusted simulator with known truth, and to check the simulator itself against real measurements (Section 2). The limit is stated plainly: benchmark accuracy numbers are on simulated boards, not real ones.

3.2 What was generated

Data set	How many	File(s)
Hidden copper maps	16 boards: 8 with 3 × 3 regions, 8 with 3 × 4 regions; each region nominal (0) or copper-rich (1), drawn at random with seed 2026	benchmark/hidden_maps.csv
Noiseless warpage fields	16 fields, 625 points each (25 × 25 grid over the 25 mm board), from the 3D finite-element model, best-fit plane removed	benchmark/ warpage_fields.npz
Noisy measurements	128 measurement sets = 16 boards × 2 measurement types (full 625-point field; 16-point 4 × 4 grid) × 4 noise levels (1, 3, 5, 10% of each board's warpage range), Gaussian noise, seed 7	benchmark/ measurements.npz; measurements_long.csv (one row per point)
FEA self-check	2 uniform boards (all nominal, all copper-rich) compared with lamination theory	benchmark/ fea_verification.json
Stress-test realities	7 boards × 5 scenarios, each with and without reference-board recalibration (Section 5)	benchmark/stress_tests.json, stress_tests_results.txt

3.3 Inputs to the generator

Parameter	Value	Source or reason
Board	25 × 25 mm; core 0.8 mm; build-up 0.1 mm (total 0.9 mm)	Scale of published flip-chip substrates (R1/R2: 31 mm, 1.08 mm). A two-layer asymmetric stack: core below, build-up on top.
Core	E 26000 MPa, ν 0.39, CTE 14.4 ppm/K	R1 Table 1 substrate row (also R2 Table 3).
Nominal build-up (ABF)	E 7500 MPa, ν 0.3, CTE 23.0 ppm/K	R7 (secondary sources); ν assumed. To confirm.
Copper	E 117000 MPa, ν 0.34, CTE 17.0 ppm/K	R8 handbook values.
Copper-rich build-up	E 40350 MPa, ν 0.312, CTE 17.78 ppm/K	Mix of ABF with 30% copper: E and ν by volume average (Voigt); CTE stiffness-weighted (Turner). The 30% copper fraction is an assumption.
Temperatures	stress-free 120 °C, measured at 25 °C (ΔT −95 K)	Zero warpage near 120 °C observed in R1; 25 °C is below Tg, so linear elasticity holds.
Solver	scikit-fem, 3D linear thermoelasticity, 27-node quadratic hexahedra, 12 × 12 elements in-plane, 2 per layer	Quadratic elements avoid shear locking in thin layers; their nodes fall exactly on the 25 × 25 output grid.
Supports	Free body, 3-2-1 constraints on bottom corners	Removes rigid-body motion without bending the board.
Output	Top-surface vertical displacement on 25 × 25 points, best-fit plane removed	The convention used for moiré warpage.
Noise	Gaussian, σ = 1, 3, 5 or 10% of the board's peak-to-valley warpage	See Table 3.4.

3.4 Why these noise levels

The generated boards warp 33–39 µm peak to valley, so the four noise levels correspond to the absolute values below. They bracket the measurement precision reported in R3 and R1.

Noise level	σ on these boards	Comparable real figure
1%	0.33–0.39 µm	Better than shadow moiré; an optimistic instrument
3%	1.0–1.2 µm	Shadow-moiré theoretical Z resolution 0.85–1.27 µm (R3)
5%	1.7–2.0 µm	Typical flatness-standard error, ~1–3 µm (R3)
10%	3.3–3.9 µm	Pessimistic; above the R3 flatness-standard error

3.5 The hidden maps

Region index = row × columns + column, with rows along x. Map 0 of each grid also serves as the reference board in the coverage and stress-test checks.

Grid	Map	Region bits
3x3	0	100100001
3x3	1	011111011
3x3	2	000111011
3x3	3	101111000
3x3	4	000011000
3x3	5	110001000
3x3	6	010010101
3x3	7	111110001
3x4	0	111001000100
3x4	1	000100100110
3x4	2	011101000010
3x4	3	100111101001
3x4	4	111010000101
3x4	5	000011100110
3x4	6	011111110001
3x4	7	000010010000

4. How the data is generated (code walkthrough)

Run order: `cd benchmark`, then `python build_benchmark.py` (data), `python run_benchmark.py` (scoring), `python summarize.py` (tables). Full listings are in the Appendix.

4.1 Materials and configuration (build_benchmark.py)

- `materials()` reads benchmark_config.json and builds the copper-rich material: Voigt average of E and ν, stiffness-weighted CTE.
- `configure(regions)` overwrites the placeholder constants at the top of fea3d.py (L = 15 mm, ΔT = −200 K and so on) with the configured geometry, ΔT = 25 − 120 = −95 K, the materials and the region grid. The fea3d.py defaults are never used for the benchmark.

4.2 Hidden maps and warpage fields (build_benchmark.py, main)

- For each grid (3 × 3, 3 × 4), 8 random 0/1 maps are drawn with `numpy.random.default_rng(2026)`.
- `fea3d.Substrate().solve(map)` returns the 625-point top-surface warpage in mm, which is converted to µm.
- The same script fits a curvature to the central region of the uniform boards and compares it with lamination theory (`clt_curvature`), giving fea_verification.json (worst difference 0.034%).

4.3 Measurements (build_benchmark.py, main)

- For every board, measurement type and noise level: σ = noise fraction × peak-to-valley of that board's field; y = field[points] + σ × standard normal, with `default_rng(7)`.
- Full field = all 625 points; sparse = 16 points at grid indices 0, 8, 16, 24 in each direction (the SPARSE array).
- One noise realisation per case, saved so every user scores against identical data.

4.4 The calibration model is deliberately different (run_benchmark.py)

- `surrogate(regions)` builds a Kirchhoff thin-plate model on the 25 × 25 grid with parameters from lamination theory of the same nominal stack. It is never fitted to the FEA output.
- `posterior(S, A, w0, y)` enumerates every map, chooses the noise level σ by maximum marginal likelihood (the true noise is not given to it), and returns the posterior.

5. Other measures and why they were taken

Measure	What it does	Why
Different physics for data and calibration	Data from 3D quadratic FEA; calibration from a 2D plate model with lamination-theory parameters	Avoids the "inverse crime" of generating test data with the same model that inverts it.
Noise level not given to the calibration	σ estimated by maximum marginal likelihood from the data	A real user does not know the exact noise; it also absorbs model mismatch.
Forward check against reality	Tsai package reproduced (Section 2)	The synthetic data is only as credible as the simulator.
Stress tests S0–S4 (stress_tests.py)	Simulated reality differs from the model: S1 true ABF CTE 25 (model 23), copper 25% (model 30%), core CTE 15 (model 14.4); S2 true map is 6 × 6 while the model stays 3 × 3; S3 measured warpage	Shows what happens when the model's assumptions are wrong. S3 and S4 sizes come from R3.

Measure	What it does	Why
	× 1.10; S4 3 µm noise plus × 1.10	
Reference board and recalibration	Board 0 has a known layout; it sizes the model error and, in the "recal" runs, fits two global factors (a, b)	How a fab would anchor a model with one characterised board.
Coverage check (coverage_and_adaptive.py)	Fraction of points where the true field lies inside the 95% prediction interval	Checks that the error bars are honest, not just the best guess.
Adaptive measurement	Start from 4 central points; repeatedly measure where plausible maps disagree most; compared with a fixed grid and random points	Tests whether fewer, smarter measurements suffice.
Fixed random seeds	Maps 2026, noise 7, stress tests 42, adaptive 3	Anyone re-running gets identical data.
Quantum settings	Speed: Layden ensemble γ ~ U[0.25, 0.6], t ~ U[2, 20]; hardware test circuits γ 0.45, t 3, 3 Trotter steps (9 qubits) and γ 0.30, t 3, 3 steps (12 qubits), started from a false local optimum	Benchmarks the method on its hardest case: escaping a false optimum.
Hardware reporting	Every run reported separately with job ID; no pooling of shots across runs; bit order checked against a noiseless run; scored against the exact ideal distribution (signal kept and XEB)	Run-to-run variation is real; pooling would hide it, and a wrong bit order could flatter a result.
Recompute script (verify_all.py)	Recomputes every hardware number from the raw counts files and prints the benchmark numbers from the saved outputs	Anyone can check that reported numbers match the files.

6. What the synthetic benchmark showed

- Calibration lowers full-field warpage error 2.8–15.2× versus the nominal design model (median per condition; results.csv).
- Exact map recovered on 8/8 boards (3 × 3) and 5/8 (3 × 4) at full field and 3% noise (summary.txt).
- 95% intervals contain the true field 92.6–100% of the time (coverage_summary.csv).
- 16 adaptive points recover 6/7 maps at 1% noise on both grids, versus 2/7 and 4/7 for a fixed grid; not enough at 3% noise or more (adaptive_summary.csv).
- Stress tests: when materials are off, the exact map is lost but the calibrated prediction stays 5–20× better than nominal with honest coverage (stress_tests_results.txt).

7. Assumptions and items to fact-check

Item	What to check	Where it is used
ABF GX-T31 properties	7.5 GPa and 23 ppm/K against the Ajinomoto datasheet; ν 0.30 is assumed	benchmark_config.json → all benchmark boards
Copper fraction 30%	Assumed value for a copper-rich region; no source	Copper-rich material (Section 3.3)
Cadence geometry	Fig. 3 and Table 3 by eye (image-only PDF); square footprints assumed	validate_flipchip.py
Tsai Fig. 13 values	Re-read the figure; values were read by eye	tsai2021_fig13_flipchip_digitized.csv
iNEMI tool-bias size	Read the package's warpage swing off R3 Fig. 11 to confirm that 20–40 µm offsets are about 10% of it	Stress tests S3 and S4
iNEMI flatness wording	Text says "~2 to 3 µm"; Table IV deviations are 1.0–2.3 µm; we summarise as ~1–3 µm	benchmark_config.json, Table 3.4
Sign convention	Negative = convex (dome) as in R1; Ansys or other	Any cross-check

Item	What to check	Where it is used
	tools may use the opposite Z direction	

Appendix: code listings

Complete source of the scripts that generate and check the data, as included in the project package (benchmark folder).

benchmark/build_benchmark.py (109 lines)

```
"""
Generate the WarpCal-Ising benchmark data from benchmark_config.json:
hidden maps, noiseless 3D FEA warpage fields, and the noisy measurements
for every task (so every user scores against identical data).
Outputs: hidden_maps.csv, warpage_fields.npz, measurements.npz,
measurements_long.csv, fea_verification.json
"""
import csv
import json
import os
import sys

import numpy as np

HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
import fea3d as F  # noqa: E402

cfg = json.load(open(os.path.join(HERE, "benchmark_config.json")))
SPARSE = np.array([i * 25 + j for i in (0, 8, 16, 24) for j in (0, 8, 16, 24)])


def materials():
    m = cfg["materials"]
    core = dict(E=m["core"]["E_MPa"], nu=m["core"]["nu"], a=m["core"]["cte_per_K"])
    nom = dict(E=m["buildup_nominal"]["E_MPa"], nu=m["buildup_nominal"]["nu"], a=m["buildup_nominal"]
["cte_per_K"])
    cu = m["copper"]
    f = m["buildup_cu_rich"]["copper_fraction"]
    E = (1 - f) * nom["E"] + f * cu["E_MPa"]
    a = ((1 - f) * nom["E"] * nom["a"] + f * cu["E_MPa"] * cu["cte_per_K"]) / E
    nu = (1 - f) * nom["nu"] + f * cu["nu"]
    return core, nom, dict(E=E, nu=nu, a=a)


def configure(regions):
    g = cfg["geometry_mm"]
    F.L, F.T_CORE, F.T_BU = g["side"], g["core_thickness"], g["buildup_thickness"]
    F.DT = cfg["thermal"]["measurement_temperature_C"] - cfg["thermal"]["zero_warpage_temperature_C"]
    F.CORE, F.BU_NOMINAL, F.BU_CU_RICH = materials()
    F.REGIONS = tuple(regions)


def main():
    os.chdir(HERE)
    t = cfg["tasks"]
    rng_maps = np.random.default_rng(t["map_seed"])
    fields, maps_all, verification = {}, {}, {}
    for rg in t["region_grids"]:
        configure(rg)
        key = f"{rg[0]}x{rg[1]}"
        R = rg[0] * rg[1]
        maps = rng_maps.integers(0, 2, (t["hidden_maps_per_grid"], R))
        sub = F.Substrate()
        sub.precompute()
        W = np.array([sub.solve(s.astype(float))[0] * 1e3 for s in maps])  # um
        fields[key], maps_all[key] = W, maps
```

```
        if key not in verification:  # FEA vs lamination theory on uniform layers
            gx = np.linspace(0, F.L, 25)
            X, Y = np.meshgrid(gx, gx, indexing="ij")
            m = ((np.abs(X - F.L / 2) < F.L * 0.3) & (np.abs(Y - F.L / 2) < F.L * 0.3)).ravel()
            x, y = X.ravel()[m] - F.L / 2, Y.ravel()[m] - F.L / 2
            M = np.column_stack([np.ones_like(x), x, y, (x ** 2 + y ** 2) / 2, x * y, x ** 2 - y ** 2])
            v = {}
            for name, s, top in [("all_nominal", np.zeros(R), F.BU_NOMINAL), ("all_cu_rich", np.ones(R),
F.BU_CU_RICH)]:
                w = sub.solve(s)[0]
                k_fea = np.linalg.lstsq(M, w[m], rcond=None)[0][3]
                k_clt = F.clt_curvature(top, F.CORE, F.T_BU, F.T_CORE, F.DT)[0]
                v[name] = dict(curvature_fea_per_mm=float(k_fea), curvature_clt_per_mm=float(k_clt),
                               relative_difference=float(abs(abs(k_fea) / abs(k_clt) - 1)),
                               warpage_peak_to_valley_um=float(np.ptp(w) * 1e3))
            verification[key] = v
        print(key, "warpage p-p um:", np.round(np.ptp(W, axis=1), 1), flush=True)

    # noisy measurements, one realisation per (grid, map, measurement, noise level)
    rng_noise = np.random.default_rng(t["noise_seed"])
    meas = {}
    rows = []
    for key in fields:
        for k, w in enumerate(fields[key]):
            rngw = np.ptp(w)
            for mname, idx in [("full_field", np.arange(625)), ("sparse_16", SPARSE)]:
                for nf in t["noise_fraction_of_warpage_range"]:
                    sig = nf * rngw
                    y = w[idx] + sig * rng_noise.standard_normal(len(idx))
                    tag = f"{key}|map{k}|{mname}|noise{nf:.2f}"
                    meas[tag] = y
                    for p, v in zip(idx, y):
                        rows.append([key, k, mname, nf, round(sig, 4), int(p), int(p // 25), int(p % 25),
round(float(v), 4)])

    with open("hidden_maps.csv", "w", newline="") as fh:
        wr = csv.writer(fh)
        wr.writerow(["grid", "map_id", "region_bits (region index = row*cols + col; row along x)"])
        for key, maps in maps_all.items():
            for k, s in enumerate(maps):
                wr.writerow([key, k, "".join(str(int(b)) for b in s)])
    np.savez_compressed("warpage_fields.npz", **{f"{k}_fields_um": v for k, v in fields.items()},
                        **{f"{k}_maps": v for k, v in maps_all.items()})
    np.savez_compressed("measurements.npz", **{k.replace("|", "__"): v for k, v in meas.items()},
                        sparse_16_indices=SPARSE)
    with open("measurements_long.csv", "w", newline="") as fh:
        wr = csv.writer(fh)
        wr.writerow(["grid", "map_id", "measurement", "noise_fraction", "noise_sigma_um", "point_index", "ix",
"iy", "warpage_um"])
        wr.writerows(rows)
    json.dump(verification, open("fea_verification.json", "w"), indent=1)
    print(json.dumps(verification, indent=1))


if __name__ == "__main__":
    main()
```

benchmark/fea3d.py (167 lines)

```
"""
Independent 3D finite-element model of thermal warpage in a two-layer package
substrate (stand-in for a shadow-moire measurement).

Geometry (mm): L x L square, glass-epoxy core at the bottom, build-up layer
```

```
(ABF dielectric + copper traces) on top. The build-up layer is split into a
3 x 3 grid of regions; each region is either "nominal" (dielectric-rich) or
"copper-rich", with different effective modulus and CTE. The substrate cools by
dT from its stress-free temperature, so CTE mismatch makes it warp.

Solver: scikit-fem, 27-node quadratic hexahedra (no shear locking in thin
layers), isotropic linear thermoelasticity, free body with 3-2-1 constraints.
Output: out-of-plane displacement of the top surface on a 25 x 25 grid, with
the best-fit plane removed (the convention used for moire warpage).
"""
import numpy as np
import scipy.sparse.linalg as spla
from skfem import Basis, BilinearForm, ElementHex2, ElementVector, LinearForm, MeshHex, asm, condense, solve
from skfem.helpers import ddot, div, sym_grad, trace

L = 15.0                # mm
T_CORE, T_BU = 0.40, 0.05  # mm (core, top build-up)
NXY = 12                # elements per side (quadratic -> 25 nodes per side = measurement grid)
DT = -200.0             # K, cool from 220 C stress-free to 20 C

# E [MPa], nu, CTE [1/K]
CORE = dict(E=25e3, nu=0.20, a=12e-6)
BU_NOMINAL = dict(E=6e3, nu=0.30, a=40e-6)
BU_CU_RICH = dict(E=15e3, nu=0.32, a=18e-6)
REGIONS = (3, 3)


def build_mesh():
    xy = np.linspace(0.0, L, NXY + 1)
    z = np.concatenate([np.linspace(0.0, T_CORE, 3), np.linspace(T_CORE, T_CORE + T_BU, 3)[1:]])
    return MeshHex.init_tensor(xy, xy, z)


def element_materials(mesh, s):
    """Per-element (E, nu, alpha) for region map s (0 = nominal, 1 = copper-rich)."""
    c = mesh.p[:, mesh.t].mean(axis=1)
    rx, ry = REGIONS
    E = np.empty(mesh.nelements)
    nu = np.empty(mesh.nelements)
    a = np.empty(mesh.nelements)
    for e in range(mesh.nelements):
        x, y, zc = c[:, e]
        if zc < T_CORE:
            m = CORE
        else:
            i = min(int(x / L * rx), rx - 1)
            j = min(int(y / L * ry), ry - 1)
            m = BU_CU_RICH if s[i * ry + j] else BU_NOMINAL
        E[e], nu[e], a[e] = m["E"], m["nu"], m["a"]
    return E, nu, a


@BilinearForm
def stiffness(u, v, w):
    eu, ev = sym_grad(u), sym_grad(v)
    return w["lam"] * trace(eu) * trace(ev) + 2.0 * w["mu"] * ddot(eu, ev)


@LinearForm
def thermal_load(v, w):
    return (3.0 * w["lam"] + 2.0 * w["mu"]) * w["a"] * DT * div(v)


class Substrate:
```

```
    def __init__(self):
        self.mesh = build_mesh()
        self.basis = Basis(self.mesh, ElementVector(ElementHex2()))
        self.nq = self.basis.X.shape[1]
        p = self.basis.doflocs
        ztop = T_CORE + T_BU
        tol = 1e-9
        # 3-2-1 constraints on bottom corners
        def node(x, y, z):
            return np.where((abs(p[0] - x) < tol) & (abs(p[1] - y) < tol) & (abs(p[2] - z) < tol))[0]
        n0, nx, ny = node(0, 0, 0), node(L, 0, 0), node(0, L, 0)
        # each returns 3 DOFs (ux, uy, uz) in doflocs order; pick by component
        comp = self._components()
        fix = list(n0) + [d for d in nx if comp[d] in (1, 2)] + [d for d in ny if comp[d] == 2]
        self.D = np.array(sorted(fix))
        # top-surface uz on the 25 x 25 measurement grid
        g = np.linspace(0.0, L, 25)
        self.obs = np.array([[d for d in node(xx, yy, ztop) if comp[d] == 2][0] for xx in g for yy in g])
        ii, jj = np.meshgrid(g, g, indexing="ij")
        B = np.column_stack([np.ones(ii.size), ii.ravel(), jj.ravel()])
        self.P_plane = np.eye(ii.size) - B @ np.linalg.pinv(B)

    def _components(self):
        comp = np.empty(self.basis.N, dtype=int)
        for k, dofs in enumerate(self.basis.nodal_dofs):
            comp[dofs] = k
        for k, dofs in enumerate(getattr(self.basis, "edge_dofs", [])):
            comp[dofs] = k
        for k, dofs in enumerate(getattr(self.basis, "facet_dofs", [])):
            comp[dofs] = k
        for k, dofs in enumerate(getattr(self.basis, "interior_dofs", [])):
            comp[dofs] = k
        return comp

    def _element_sets(self):
        """Element index sets: core, then the 9 build-up regions."""
        c = self.mesh.p[:, self.mesh.t].mean(axis=1)
        rx, ry = REGIONS
        reg = np.full(self.mesh.nelements, -1)
        top = c[2] > T_CORE
        i = np.minimum((c[0] / L * rx).astype(int), rx - 1)
        j = np.minimum((c[1] / L * ry).astype(int), ry - 1)
        reg[top] = (i * ry + j)[top]
        return [np.where(reg == r)[0] for r in range(-1, rx * ry)]

    def precompute(self, cache=None):
        """K is linear in (lambda, mu) per element set and f is linear in
        (3 lambda + 2 mu) alpha dT, so assemble each set once and recombine."""
        import os
        cache = cache or "fea_blocks_%dx%d_L%g_c%g_b%g.npz" % (REGIONS + (L, T_CORE, T_BU))
        if os.path.exists(cache):
            z = np.load(cache, allow_pickle=True)
            self.KL, self.KM = list(z["KL"]), list(z["KM"])
            self.FV = [np.asarray(v, dtype=float) for v in z["FV"]]
            return
        unit_l = BilinearForm(lambda u, v, w: trace(sym_grad(u)) * trace(sym_grad(v)))
        unit_m = BilinearForm(lambda u, v, w: 2.0 * ddot(sym_grad(u), sym_grad(v)))
        unit_f = LinearForm(lambda v, w: div(v))
        self.KL, self.KM, self.FV = [], [], []
        for els in self._element_sets():
            b = Basis(self.mesh, ElementVector(ElementHex2()), elements=els)
            self.KL.append(asm(unit_l, b).tocsr())
            self.KM.append(asm(unit_m, b).tocsr())
            self.FV.append(asm(unit_f, b))
```

```
        np.savez(cache, KL=np.array(self.KL, dtype=object), KM=np.array(self.KM, dtype=object),
                 FV=np.array(self.FV, dtype=object))

    def solve(self, s):
        if not hasattr(self, "KL"):
            self.precompute()
        assert len(s) == REGIONS[0] * REGIONS[1]
        mats = [CORE] + [BU_CU_RICH if v else BU_NOMINAL for v in s]
        K = 0
        f = 0
        for m, KL, KM, FV in zip(mats, self.KL, self.KM, self.FV):
            lam = m["E"] * m["nu"] / ((1 + m["nu"]) * (1 - 2 * m["nu"]))
            mu = m["E"] / (2 * (1 + m["nu"]))
            K = K + lam * KL + mu * KM
            f = f + (3 * lam + 2 * mu) * m["a"] * DT * FV
        u = solve(*condense(K.tocsr(), f, D=self.D),
                  solver=lambda A, b: spla.spsolve(A.tocsc(), b, permc_spec="MMD_AT_PLUS_A"))
        uz = u[self.obs]
        return self.P_plane @ uz, u


def clt_curvature(top, bottom=CORE, t_top=T_BU, t_bot=T_CORE, dT=DT):
    """Classical lamination theory curvature of a free, uniform bilayer (equibiaxial)."""
    zs = [(0.0, t_bot, bottom), (t_bot, t_bot + t_top, top)]
    A = B = Dm = Nt = Mt = 0.0
    for z0, z1, m in zs:
        Q = m["E"] / (1 - m["nu"])           # biaxial modulus
        A += Q * (z1 - z0)
        B += Q * (z1 ** 2 - z0 ** 2) / 2
        Dm += Q * (z1 ** 3 - z0 ** 3) / 3
        Nt += Q * m["a"] * dT * (z1 - z0)
        Mt += Q * m["a"] * dT * (z1 ** 2 - z0 ** 2) / 2
    eps0, kappa = np.linalg.solve([[A, B], [B, Dm]], [Nt, Mt])
    return kappa, Dm - B ** 2 / A
```

benchmark/run_benchmark.py (166 lines)

```
"""
Score every benchmark task with the calibration pipeline and classical /
quantum samplers. Writes results.csv and summary.json.
Quantum sampler speed is computed for all 3x3 tasks and for one pre-declared
3x4 slice (sparse_16, 3% noise), since exact 4,096-state gaps are slow.
"""
import csv
import json
import os
import sys
import time

import numpy as np
from scipy.optimize import lsq_linear

HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)

import build_benchmark as B  # noqa: E402
import fea3d as F  # noqa: E402
import warpage_ising_check as W  # noqa: E402

# Proposal fidelity f used in the noise model Q_f = f*Q + (1-f)*Uniform. Values are signal-kept
# levels from the 9-qubit hardware test (hardware_9q): IonQ 0.84 = mean of the two Forte-1 runs
# (0.98, 0.71; a later Forte Enterprise run gave 0.60), IBM 0.30 = Kingston run 1 (Fez 0.38,
# Marrakesh 0.13, Kingston run 2 0.11), Rigetti 0.10 = Cepheus. They are model inputs, not results.
FIDELITY = {"ionq": 0.84, "ibm": 0.30, "rigetti": 0.10}
```

```
QUANTUM_3X4_SLICE = ("sparse_16", 0.03)


def surrogate(regions):
    B.configure(regions)
    k0, D0 = F.clt_curvature(F.BU_NOMINAL, F.CORE, F.T_BU, F.T_CORE, F.DT)
    k1, D1 = F.clt_curvature(F.BU_CU_RICH, F.CORE, F.T_BU, F.T_CORE, F.DT)
    dD = D1 / D0 - 1
    prm = dict(m0=k0, dm=k1 * (1 + dD) - k0, D0=1.0, dD=dD)
    plate = W.Plate(25, tuple(regions))
    h = F.L / 24
    R = regions[0] * regions[1]
    f = lambda s: plate.solve(np.asarray(s, float), **prm) * h ** 2 * 1e3
    w0 = f(np.zeros(R))
    A = np.column_stack([f(np.eye(R)[i]) - w0 for i in range(R)])
    return w0, A


def posterior(S, A, w0, y):
    sse = np.sum((S @ A.T + w0 - y) ** 2, 1)
    best = None
    for sig in np.geomspace(0.05, 50, 200):
        E = sse / (2 * sig ** 2)
        ll = -E.min() + np.log(np.exp(-(E - E.min())).sum()) - len(y) * np.log(sig)
        if best is None or ll > best[0]:
            best = (ll, sig)
    E = sse / (2 * best[1] ** 2)
    E -= E.min()
    p = np.exp(-E)
    return E, p / p.sum(), best[1]


def neighbours(M, R):
    return np.arange(M)[:, None] ^ (1 << np.arange(R))[None, :]


def annealing(E, R, rng, runs=200, flips=2000):
    M = len(E)
    nb = neighbours(M, R)
    x = rng.integers(M, size=runs)
    best = x.copy()
    for t in range(flips):
        T = 5.0 * (0.01 / 5.0) ** (t / flips)
        y = nb[x, rng.integers(R, size=runs)]
        acc = rng.random(runs) < np.exp(np.minimum(0.0, (E[x] - E[y]) / T))
        x = np.where(acc, y, x)
        best = np.where(E[x] < E[best], x, best)
    return float(np.mean(best == np.argmin(E)))


def quantum_Q(E, R, rng, n_gamma=4, n_t=2):
    M = len(E)
    S = ((np.arange(M)[:, None] >> np.arange(R)) & 1).astype(float)
    Z = 2 * S - 1
    X = np.column_stack([np.ones(M)] + [Z[:, i] for i in range(R)] +
                        [Z[:, i] * Z[:, j] for i in range(R) for j in range(i + 1, R)])
    c = np.linalg.lstsq(X, E, rcond=None)[0]
    alpha = np.sqrt(R) / np.sqrt(np.sum(c[1:] ** 2))
    Hmix = np.zeros((M, M))
    for k in range(R):
        Hmix[np.arange(M), np.arange(M) ^ (1 << k)] = 1.0
    Q = np.zeros((M, M))
    for _ in range(n_gamma):
        g = rng.uniform(0.25, 0.6)
```

```
        vals, vecs = np.linalg.eigh(np.diag((1 - g) * alpha * (E - E.mean())) + g * Hmix)
        for _ in range(n_t):
            t = rng.uniform(2, 20)
            U = (vecs * np.exp(-1j * vals * t)) @ vecs.conj().T
            Q += np.abs(U) ** 2
    Q /= n_gamma * n_t
    return 0.5 * (Q + Q.T)


def local_Q(R):
    M = 2 ** R
    Q = np.zeros((M, M))
    for k in range(R):
        Q[np.arange(M), np.arange(M) ^ (1 << k)] = 1.0 / R
    return Q


def main():
    os.chdir(HERE)
    cfg = B.cfg
    meas = np.load("measurements.npz")
    fields = np.load("warpage_fields.npz")
    rng = np.random.default_rng(11)
    rows = []
    for rg in cfg["tasks"]["region_grids"]:
        key = f"{rg[0]}x{rg[1]}"
        R = rg[0] * rg[1]
        M = 2 ** R
        w0_full, A_full = surrogate(rg)
        S = ((np.arange(M)[:, None] >> np.arange(R)) & 1).astype(float)
        Uq = np.full((M, M), 1.0 / M)
        Lq = local_Q(R)
        for k, (wf, s_true) in enumerate(zip(fields[f"{key}_fields_um"], fields[f"{key}_maps"])):
            kt = int(sum(int(v) << i for i, v in enumerate(s_true)))
            for mname, idx in [("full_field", np.arange(625)), ("sparse_16", B.SPARSE)]:
                for nf in cfg["tasks"]["noise_fraction_of_warpage_range"]:
                    t0 = time.time()
                    y = meas[f"{key}__map{k}__{mname}__noise{nf:.2f}"]
                    A, w0 = A_full[idx], w0_full[idx]
                    E, p, sig = posterior(S, A, w0, y)
                    kmap = int(np.argmax(p))
                    pred = p @ (S @ A_full.T + w0_full)
                    rms = lambda e: float(np.sqrt(np.mean(e ** 2)))
                    s_ls = np.round(lsq_linear(A, y - w0, bounds=(0, 1)).x)
                    nmin = int(np.sum([np.all(E[np.arange(M)[i] ^ (1 << np.arange(R))] >= E[i]) for i in
range(M)]))
                    row = dict(grid=key, map_id=k, measurement=mname, noise_fraction=nf,
                               noise_sigma_um=round(nf * np.ptp(wf), 4), fitted_sigma_um=round(float(sig), 4),
                               map_regions_correct=int((S[kmap] == s_true).sum()), regions=R,
                               p_true_map=round(float(p[kt]), 4), rank_true_map=int((p > p[kt]).sum()) + 1,
                               maps_for_90pct=int(np.searchsorted(np.cumsum(np.sort(p)[::-1]), 0.9) + 1),
                               local_optima=nmin,
                               nominal_rms_um=round(rms(w0_full - wf), 3), calibrated_rms_um=round(rms(pred -
wf), 3),
                               lsq_round_regions_correct=int((s_ls == s_true).sum()),
                               sa_success=annealing(E, R, rng))
                    if R == 9 or (mname, nf) == QUANTUM_3X4_SLICE:
                        Qq = quantum_Q(E, R, rng)
                        g_loc, g_uni = W.spectral_gap(E, Lq), W.spectral_gap(E, Uq)
                        best = max(g_loc, g_uni)
                        row.update(gap_single_flip_vs_best=g_loc / best, gap_uniform_vs_best=g_uni / best,
                                   gap_quantum_ideal_vs_best=W.spectral_gap(E, Qq) / best)
                        for name, f in FIDELITY.items():
```

```
                            row[f"gap_quantum_{name}_vs_best"] = W.spectral_gap(E, f * Qq + (1 - f) * Uq) /
best
                    rows.append(row)
                    print(key, k, mname, nf, row["map_regions_correct"], row["calibrated_rms_um"],
                          round(row.get("gap_quantum_ionq_vs_best", float("nan")), 2), f"{time.time() -
t0:.1f}s", flush=True)
    cols = sorted({c for r in rows for c in r}, key=lambda c: list(rows[-1]).index(c) if c in rows[-1] else 99)
    cols = list(dict.fromkeys(list(rows[0]) + [c for r in rows for c in r]))
    with open("results.csv", "w", newline="") as fh:
        wr = csv.DictWriter(fh, fieldnames=cols)
        wr.writeheader()
        wr.writerows(rows)


if __name__ == "__main__":
    main()
```

benchmark/validate_flipchip.py (97 lines)

```
"""
External check of the 3D FEA against a published shadow-moire measurement:
flip-chip package from Tsai, Wang & Liu, Materials 14:3723 (2021), as modelled in
the Cadence white paper "Thermal Warpage Simulation of a Temperature-Dependent
Linear Elastic Material Package" (Fig. 3 geometry, Table 3 materials, Fig. 6 data).

Geometry (mm): substrate 31 x 31 x 1.08; underfill/bump layer 10 x 10 x 0.07;
die 10 x 10 x 0.785, centred. Square footprints assumed (the figure gives one side).
Materials below Tg (Table 3): die 131 GPa / 0.28 / 2.8 ppm; underfill 7.6 GPa / 0.32 /
29 ppm; third column (presumably substrate) 26 GPa / 0.39 / 14.4 ppm.
Zero warpage at 120 C (shadow moire) -> stress-free 120 C; evaluate at 25 C.
Published corner warpage at 25 C: about -57 um (read off Cadence Fig. 6; simulation
and both moire curves coincide there).
"""
import numpy as np
import scipy.sparse.linalg as spla
from skfem import Basis, BilinearForm, ElementHex2, ElementVector, LinearForm, MeshHex, asm, condense, solve
from skfem.helpers import ddot, div, sym_grad, trace

L, DIE = 31.0, 10.0
TS, TU, TD = 1.08, 0.07, 0.785
a0, a1 = (L - DIE) / 2, (L + DIE) / 2
xs = np.unique(np.concatenate([np.linspace(0, a0, 6), np.linspace(a0, a1, 6), np.linspace(a1, L, 6)]))
zs = np.unique(np.concatenate([np.linspace(0, TS, 3), [TS + TU], np.linspace(TS + TU, TS + TU + TD, 3)]))
mesh = MeshHex.init_tensor(xs, xs, zs)
basis = Basis(mesh, ElementVector(ElementHex2()))
c = mesh.p[:, mesh.t].mean(axis=1)
inside = (c[0] > a0) & (c[0] < a1) & (c[1] > a0) & (c[1] < a1)
MAT = {"sub": (26e3, 0.39, 14.4e-6), "uf": (7.6e3, 0.32, 29e-6), "die": (131e3, 0.28, 2.8e-6), "void": (1e-3,
0.0, 0.0)}
mat = np.where(c[2] < TS, "sub", np.where(~inside, "void", np.where(c[2] < TS + TU, "uf", "die")))
E = np.array([MAT[m][0] for m in mat]); nu = np.array([MAT[m][1] for m in mat]); al = np.array([MAT[m][2] for m
in mat])
nq = basis.X.shape[1]
rep = lambda q: np.repeat(q[:, None], nq, axis=1)
DT = 1.0  # unit temperature change; linear elasticity lets us scale


@BilinearForm
def k(u, v, w):
    return w["lam"] * trace(sym_grad(u)) * trace(sym_grad(v)) + 2 * w["mu"] * ddot(sym_grad(u), sym_grad(v))


@LinearForm
def f(v, w):
```

```
    return (3 * w["lam"] + 2 * w["mu"]) * w["a"] * DT * div(v)



p = basis.doflocs
comp = np.empty(basis.N, int)
for attr in ("nodal_dofs", "edge_dofs", "facet_dofs", "interior_dofs"):
    for kk, d in enumerate(getattr(basis, attr, [])):
        comp[d] = kk
node = lambda x, y, z: np.where((abs(p[0] - x) < 1e-9) & (abs(p[1] - y) < 1e-9) & (abs(p[2] - z) < 1e-9))[0]
n0, nx, ny = node(0, 0, 0), node(L, 0, 0), node(0, L, 0)
D = np.array(sorted(list(n0) + [d for d in nx if comp[d] in (1, 2)] + [d for d in ny if comp[d] == 2]))
diag = [(t, t) for t in np.unique(xs) if t >= L / 2]


# K and the thermal load are linear in (lambda, mu, alpha) per material set: assemble once
unit_l = BilinearForm(lambda u, v, w: trace(sym_grad(u)) * trace(sym_grad(v)))
unit_m = BilinearForm(lambda u, v, w: 2.0 * ddot(sym_grad(u), sym_grad(v)))
unit_f = LinearForm(lambda v, w: div(v))
BLOCKS = {}
for name in ("sub", "uf", "die", "void"):
    b = Basis(mesh, ElementVector(ElementHex2()), elements=np.where(mat == name)[0])
    BLOCKS[name] = (asm(unit_l, b).tocsr(), asm(unit_m, b).tocsr(), asm(unit_f, b))


def unit_response(uf):
    MAT["uf"] = uf
    K, Fv = 0, 0
    for name, (KL, KM, FV) in BLOCKS.items():
        E_, nu_, a_ = MAT[name]
        lam_, mu_ = E_ * nu_ / ((1 + nu_) * (1 - 2 * nu_)), E_ / (2 * (1 + nu_))
        K = K + lam_ * KL + mu_ * KM
        Fv = Fv + (3 * lam_ + 2 * mu_) * a_ * DT * FV
    u = solve(*condense(K, Fv, D=D), solver=lambda A, b: spla.spsolve(A.tocsc(), b))
    uz = lambda x, y: u[[d for d in node(x, y, 0.0) if comp[d] == 2][0]]
    centre = uz(L / 2, L / 2)
    corners = np.mean([uz(L, L), uz(0, 0), uz(L, 0), uz(0, L)]) - centre
    prof = [(np.sqrt(2) * (x - L / 2), uz(x, y) - centre) for x, y in diag]
    return corners * 1e3, prof  # um per K


below, prof_b = unit_response((7.6e3, 0.32, 29e-6))
above, prof_a = unit_response((0.1e3, 0.32, 98e-6))  # underfill above Tg: Tsai et al. 2021, Table 1 (100 MPa,
98 ppm/K)
T = [25, 40, 60, 80, 100, 120, 140, 160, 180, 200, 220, 240, 260]
moire_heat = [-58, -50, -38, -28, -18, 0, 12, 12, 14, 17, 19, 22, 27]
fem_paper = [-58, -49, -37, -24, -12, 0, 4, 8, 12, 16, 20, 24, 28]
ours = [below * (t - 120) if t <= 120 else above * (t - 120) for t in T]
print("T(C)  ours  paperFEM  moire(heating)   [um, corner c relative to centre; paper values read off Fig.13]")
for t, a, b2, c2 in zip(T, ours, fem_paper, moire_heat):
    print(f"{t:4d} {a:6.1f} {b2:6d} {c2:6d}")
o, m = np.array(ours), np.array(moire_heat)
print("RMS ours vs moire (heating):", round(float(np.sqrt(np.mean((o - m) ** 2))), 1), "um over a", int(m.max()
- m.min()), "um range")
print("diagonal profile at 25 C (x mm, ours um):", [(round(x, 1), round(w * 1e3 * -95, 1)) for x, w in prof_b])
print("diagonal profile at 260 C (x mm, ours um):", [(round(x, 1), round(w * 1e3 * 140, 1)) for x, w in
prof_a])
```

benchmark/stress_tests.py (133 lines)

```
"""
Stress tests against the "inverse crime" (data generated under the same
idealisations the calibration assumes). The calibration model, its assumed
materials and its 3 x 3 grid are NOT changed; only the simulated reality is.
```

```
S0  baseline: benchmark reality, 3% noise
S1  materials off: true ABF CTE 25 ppm/K (model assumes 23), true copper fraction
    25% (model 30%), true core CTE 15 ppm/K (model 14.4)
S2  finer truth: reality has a 6 x 6 binary copper map; the model still uses 3 x 3
S3  tool bias: measured warpage scaled by 1.10 (iNEMI reports 20-40 um tool-to-tool
    differences on ~250 um swings)
S4  iNEMI-level noise: 3 um per point plus the 1.10 scale bias

Map 0 is the reference board (known layout, measured under the same conditions)
used only to size model discrepancy. Maps 1-7 are scored.
"""
import json

import numpy as np
from scipy.special import ndtr

import build_benchmark as B
import fea3d as F
import run_benchmark as RB

rng = np.random.default_rng(42)
R = 9
S = ((np.arange(512)[:, None] >> np.arange(R)) & 1).astype(float)
w0f, Af = RB.surrogate((3, 3))          # calibration model (assumed materials, 3 x 3 grid)
P = S @ Af.T + w0f
maps9 = np.load("warpage_fields.npz")["3x3_maps"]


def reality(regions, maps, materials=None):
    B.configure(regions)
    if materials:
        F.CORE, F.BU_NOMINAL, F.BU_CU_RICH = materials
    sub = F.Substrate()
    sub.precompute()
    return np.array([sub.solve(np.asarray(s, float))[0] * 1e3 for s in maps])


def materials_off():
    m = B.cfg["materials"]
    core = dict(E=m["core"]["E_MPa"], nu=m["core"]["nu"], a=15e-6)
    nom = dict(E=7500, nu=0.30, a=25e-6)
    f, cu = 0.25, m["copper"]
    E = (1 - f) * nom["E"] + f * cu["E_MPa"]
    rich = dict(E=E, nu=(1 - f) * 0.30 + f * cu["nu"], a=((1 - f) * nom["E"] * nom["a"] + f * cu["E_MPa"] *
cu["cte_per_K"]) / E)
    return core, nom, rich


def calibrate(y, sig_noise, sig_ref):
    E, p, _ = RB.posterior(S, Af, w0f, y)
    sig = np.sqrt(sig_noise ** 2 + sig_ref ** 2)
    E = np.sum((P - y) ** 2, 1) / (2 * sig ** 2)
    p = np.exp(-(E - E.min()))
    return p / p.sum()


def coverage(p, w_true, s):
    keep = p > 1e-8
    pk, Pk = p[keep] / p[keep].sum(), P[keep]
    Fc = pk @ ndtr((w_true[None, :] - Pk) / max(s, 1e-9))
    return float(np.mean((Fc > 0.025) & (Fc < 0.975)))
```

```
def smooth_rms(resid, sig_noise, k=5):
    """RMS of the smooth part of a residual field: k x k moving average, noise variance removed."""
    g = resid.reshape(25, 25)
    pad = np.pad(g, k // 2, mode="edge")
    sm = np.mean([pad[i:i + 25, j:j + 25] for i in range(k) for j in range(k)], axis=0)
    return float(np.sqrt(max(np.mean(sm ** 2) - sig_noise ** 2 / k ** 2, 0)))


def run(name, W_true, truth9, noise_frac=None, noise_um=None, scale=1.0, recal=False):
    global P
    P_base = S @ Af.T + w0f
    P = P_base
    rows = []
    sig_n = lambda w: noise_um if noise_um else noise_frac * np.ptp(w)
    # reference board 0: known layout -> model discrepancy (in excess of noise) under these conditions
    w_ref = W_true[0]
    y_ref = scale * w_ref + sig_n(w_ref) * rng.standard_normal(625)
    k_ref = int(sum(int(v) << i for i, v in enumerate(truth9[0]))) if truth9 is not None else None
    if k_ref is None:  # finer truth: best 3 x 3 fit to the reference board stands in for "known layout"
        k_ref = int(np.argmin(np.mean((P - y_ref) ** 2, 1)))
    if recal:  # fit two global factors on the reference board: y ~ a*w0 + b*(A s_ref)
        X = np.column_stack([w0f, Af @ S[k_ref]])
        a, b = np.linalg.lstsq(X, y_ref, rcond=None)[0]
        P = a * w0f + b * (S @ Af.T)
    sig_ref = smooth_rms(y_ref - P[k_ref], sig_n(w_ref))
    for k in range(1, len(W_true)):
        w = W_true[k]
        y = scale * w + sig_n(w) * rng.standard_normal(625)
        p = calibrate(y, sig_n(w), sig_ref)
        pred = p @ P / scale if recal else p @ P
        r = dict(rms_nominal=float(np.sqrt(np.mean((w0f - w) ** 2))), rms_cal=float(np.sqrt(np.mean((pred - w)
** 2))),
                 coverage95=coverage(p, scale * w if recal else w, sig_ref))
        if truth9 is not None:
            kmap = int(np.argmax(p))
            r.update(map_exact=bool((S[kmap] == truth9[k]).all()), regions_correct=int((S[kmap] ==
truth9[k]).sum()))
        rows.append(r)
    agg = {key: float(np.median([r[key] for r in rows])) for key in rows[0] if key != "map_exact"}
    if truth9 is not None:
        agg["maps_exact"] = f"{sum(r['map_exact'] for r in rows)}/{len(rows)}"
    agg["model_error_from_reference_um"] = sig_ref
    if recal:
        agg["recal_factors"] = [round(float(a), 3), round(float(b), 3)]
    print(f"{name:34s}", {k2: (round(v, 3) if isinstance(v, float) else v) for k2, v in agg.items()},
flush=True)
    return agg


def main():
    out = {}
    W0 = np.load("warpage_fields.npz")["3x3_fields_um"]
    W1 = reality((3, 3), maps9, materials_off())
    maps36 = rng.integers(0, 2, (8, 36))
    W2 = reality((6, 6), maps36)
    np.savez("stress_fields.npz", W1=W1, W2=W2, maps36=maps36)
    for rc in (False, True):
        tag = " + reference-board recal" if rc else ""
        for nm, args in [("S0 baseline (3% noise)", (W0, maps9, dict(noise_frac=0.03))),
                         ("S1 materials 10-20% off", (W1, maps9, dict(noise_frac=0.03))),
                         ("S2 finer 6x6 truth", (W2, None, dict(noise_frac=0.03))),
                         ("S3 tool bias x1.10", (W0, maps9, dict(noise_frac=0.03, scale=1.10))),
                         ("S4 iNEMI noise 3um + bias", (W0, maps9, dict(noise_um=3.0, scale=1.10)))]:
            out[nm + tag] = run(nm + tag, args[0], args[1], recal=rc, **args[2])
```

```
    json.dump(out, open("stress_tests.json", "w"), indent=1)


if __name__ == "__main__":
    main()
```